

Serie de ejercicios – AGM, minimax y maximin (Python)

Objetivo

Práctica compacta para fijar AGMs (MST), caminos minimax y maximin. Cada enunciado indica objetivo, entrada/salida y complejidad objetivo. No incluye soluciones.

Convenciones

- Grafo no dirigido; vértices $0..n-1$.
- edges: list[(u,v,w)] con $u \neq v$ y peso w (int).
- adj: list[list[(v,w)]] (no dirigido).
- Usar DSU (union-find), heapq y listas de adyacencia.

1) Kruskal con DSU (bosque generador mínimo si hay varias componentes)

Implementá: `kruskal_mst(n, edges)`

Entrada: n: int; edges: list[(u,v,w)].

Salida: (mst_weight: int, mst_edges: list[(u,v,w)]).

Requisitos: ordenar por (w,u,v); DSU con path compression + union by rank.

Complejidad objetivo: $O(m \log m + m \alpha(n))$.

2) Prim con heap binario (grafo conexo)

Implementá: `prim_mst(n, adj)`

Entrada: adj: list[list[(v,w)]], n: int.

Salida: (mst_weight: int, parent: list[int]) con `parent[root]=-1`.

Requisitos: `root=0`; decrease-key por reinserción (tuplas (key, v)).

Complejidad objetivo: $O(m \log n)$.

3) MST del grafo completo de secuencias (distancia L1)

Implementá: `mst_sequences(X)`

Entrada: X: list[list[int]] (n secuencias de largo k).

Peso: $d(X_i, X_j) = \sum(|x_{i\ell} - x_{j\ell}|)$.

Salida: (mst_weight, mst_edges).

Requisitos: generar aristas on-the-fly (doble for); Kruskal.

Complejidad objetivo: $O(k n^2 + n^2 \log n)$.

4) Árbol bottleneck (MBST)

Implementá: `bottleneck_spanning_tree(n, edges)`

Entrada/Salida: igual a Kruskal.

Objetivo: devolver un spanning tree que minimiza su arista máxima (bottleneck).

Pista: cualquier MST es MBST; el bottleneck es la última arista del MST.

Complejidad objetivo: $O(m \log m)$.

5) Árbol generador máximo (MaxST) y valor maximin $s \rightarrow t$

Implementá: `maximum_spanning_tree(n, edges)` y `maximin_path_value(n, edges, s, t)`

Entrada: `edges; s, t`.

Salida: MaxST: (weight, edges). Maximin: entero = max sobre caminos de min-arista.

Pista: en el MaxST, el mínimo a lo largo del camino $s-t$ da el valor maximin.

Complejidad objetivo: construir $O(m \log m)$; consulta $s-t$ $O(\text{largo del camino})$ o $O(\log n)$ con LCA+RMQ(min).

6) Valor minimax $s \rightarrow t$ (minimizar el máximo en el camino)

Implementá: `minimax_path_value(n, edges, s, t)`

Estrategias válidas (elegí una):

A) MST mínimo + máximo del camino $s-t$ en el MST.

B) Binary search sobre W y DSU/BFS con aristas $\leq$ umbral.

Salida: entero.

Complejidad objetivo: A) $O(m \log m + n)$. B) $O(m \log m + (m+n) \log m)$.

7) Network bandwidth (umbral máximo k que mantiene el grafo conexo)

Implementá: `network_bandwidth(n, edges)`

Idea: ordenar aristas por peso decreciente y unir con DSU; el k^* es el peso que conecta todo.

Complejidad objetivo: $O(m \log m + m \alpha(n))$.

8) Vector de ancho de banda con i upgrades ($a[i]$, $i=0..n-1$)

Implementá: `bandwidth_after_upgrades(n, edges)`

Significado: $a[i] = \text{máximo } k \text{ tal que } G_k \text{ puede hacerse conexo si actualizás } i \text{ aristas a "infinito"}$.

Pista: procesar por bloques de peso decreciente con DSU; registrar #componentes $c(k)$. $a[i] = \max\{k : c(k) \leq i+1\}$. Llenar el vector de mayor a menor peso; $a[n-1] = +\infty$ o $\geq \max \text{ peso}$.

Complejidad objetivo: $O(m \log m + n)$.

9) Borůvka (versión simple)

Implementá: `boruvka_mst(n, edges)`

Idea: por fase, para cada componente elegir su arista saliente más barata y contraer. Repetir hasta 1 componente.

Complejidad objetivo: $O(m \log n)$ aprox. (o $O(m \alpha(n) \log n)$ según DSU/representación).

10) Unicidad del MST

Implementá: `is_mst_unique(n, edges) -> bool`

Criterios:

- Si todos los pesos son distintos $\Rightarrow$ único.
- General: corré Kruskal; para cada arista fuera del MST, chequeá si el máximo en el camino del MST (LCA+RMQ(max)) tiene el mismo peso $\Rightarrow$ no único.

Complejidad objetivo: $O(m \log m + n \log n + m \log n)$.

Notas finales

- Para consultas múltiples de minimax o maximin, preprocesá LCA con binary lifting guardando en cada salto el max (minimax, sobre MST) o el min (maximin, sobre MaxST).
- Validar entradas: si el grafo no es conexo y la función requiere conectividad, devolver error claro o bosque.